```
# in app/controllers/admin/products_controller
class Admin::ProductsController < AdminController
  def index
  end
end
```

The lookup order for a `admin/products#index` action will be:

- `app/views/admin/products/`
- `app/views/admin/`
- `app/views/application/`

This makes `app/views/application/` a great place for your shared partials, which can then be rendered in your ERb as such:

```
<%# app/views/admin/products/index.html.erb %>
<%= render @products || "empty_list" %>

<%# app/views/application/_empty_list.html.erb %>
There are no items in this list <em>yet</em>.
```

2.2.15 Avoiding Double Render Errors

Sooner or later, most Rails developers will see the error message "Can only render or redirect once per action". While this is annoying, it's relatively easy to fix. Usually it happens because of a fundamental misunderstanding of the way that `render` works.

For example, here's some code that will trigger this error:

```
def show
  @book = Book.find(params[:id])
  if @book.special?
    render action: "special_show"
  end
  render action: "regular_show"
end
```

If `@book.special?` evaluates to `true`, Rails will start the rendering process to dump the `@book`variable into the `special_show` view. But this will *not* stop the rest of the code in the `show` action from running, and when Rails hits the end of the action, it will start to render the `regular_show` view - and throw an error. The solution is simple: make sure that you have only one call to `render` or `redirect`in a single code path. One thing that can help is `and return`. Here's a patched version of the method:

```
def show
  @book = Book.find(params[:id])
  if @book.special?
    render action: "special_show" and return
  end
  render action: "regular_show"
end
```

Make sure to use `and return` instead of `&& return` because `&& return` will not work due to the operator precedence in the Ruby Language.

Note that the implicit render done by ActionController detects if `render` has been called, so the following will work without errors:

```
def show
  @book = Book.find(params[:id])
  if @book.special?
    render action: "special_show"
  end
end
```

This will render a book with `special?` set with the `special_show` template, while other books will render with the default `show` template.